

AI & Agent Tooling Stack

Local Model Hosting, LLM Routing, Browser Automation, Multi-Agent Orchestration

Marouane Aabirrouche — IT Infrastructure specialist — Réseaux & Systèmes
Project write-up PRJ-07 • work in progress — living documentation

Status note

This node groups four small, individually undocumented projects into one infrastructure-and-orchestration story: *the tooling layer behind how I build*. It is deliberately presented as in-progress rather than a finished, shipped product.

1 Overview

Rather than treating AI tooling as a set of demos, this stack treats it as an infrastructure problem: models are workloads with hardware constraints, requests need a routing layer with policy, browsers become automatable endpoints, and complex tasks decompose into agent teams with explicit delegation rules.

2 Local AI hosting and quantization

Consumer hardware sets real constraints: an 8 GB VRAM budget (RTX 2060 Super) with 32 GB of system RAM. The work here is making useful models fit that box:

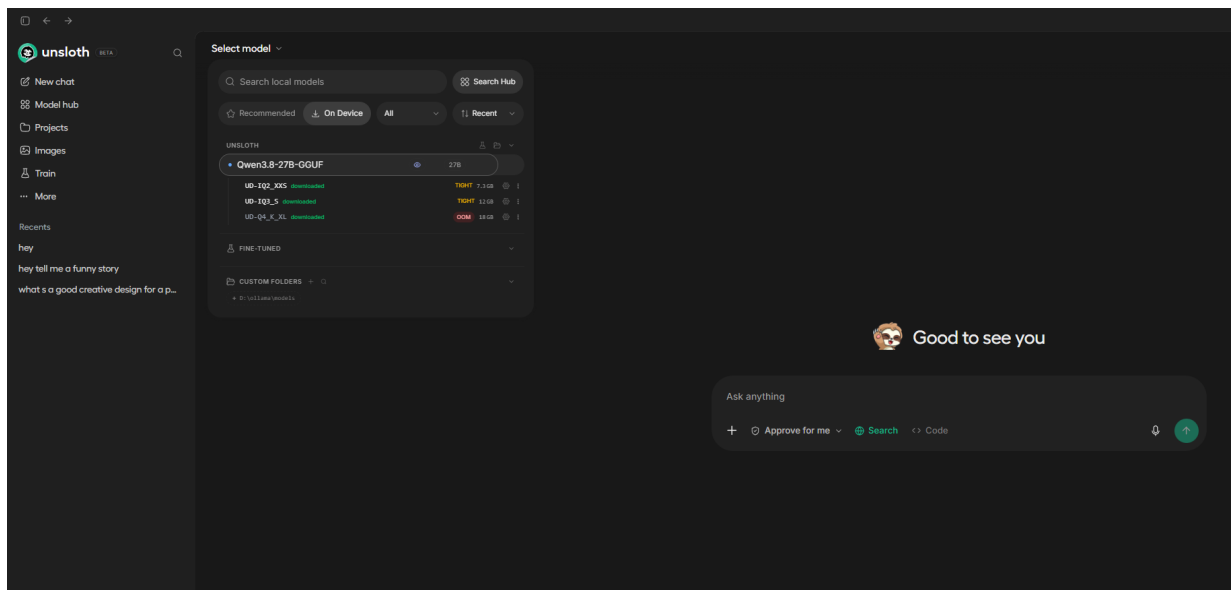


Figure 1: Unsloth interface hosting local quantized models.

- Running **quantized** and smaller model variants so weights fit in VRAM while activations spill gracefully to system RAM.
- Evaluating **Mixture-of-Experts** architectures, whose sparse activation makes them unusually friendly to tight memory budgets — explored via **llama.cpp**, **ExLlamaV2**/TabbyAPI, and **Ollama**, each with different trade-offs in speed, memory layout and serving ergonomics.

3 OmniRoute — self-hosted LLM routing layer

OmniRoute is the piece that turns local inference into infrastructure. It runs locally on `localhost:20128` and exposes an **OpenAI-compatible API**, so any client that can talk to OpenAI can talk to the whole local fleet without knowing what is behind it.

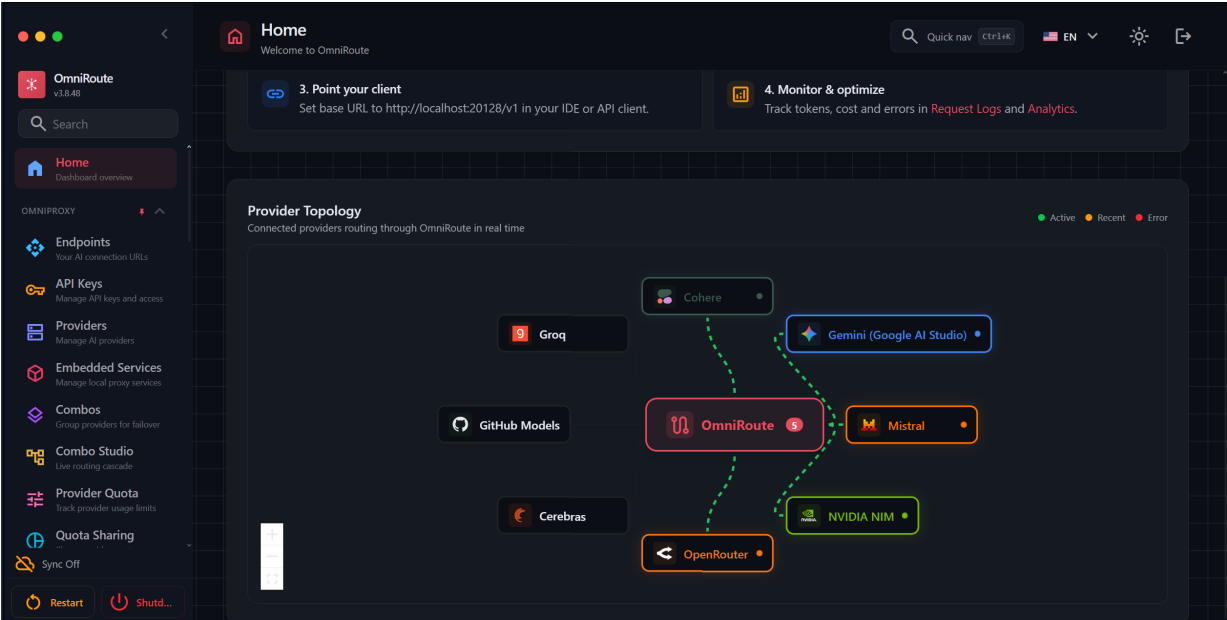


Figure 2: OmniRoute Provider Topology mapping connected backends.

Requests are routed across named model pools, each tuned for a job family:

planning	coding	review/security	vision	cheap
----------	--------	-----------------	--------	-------

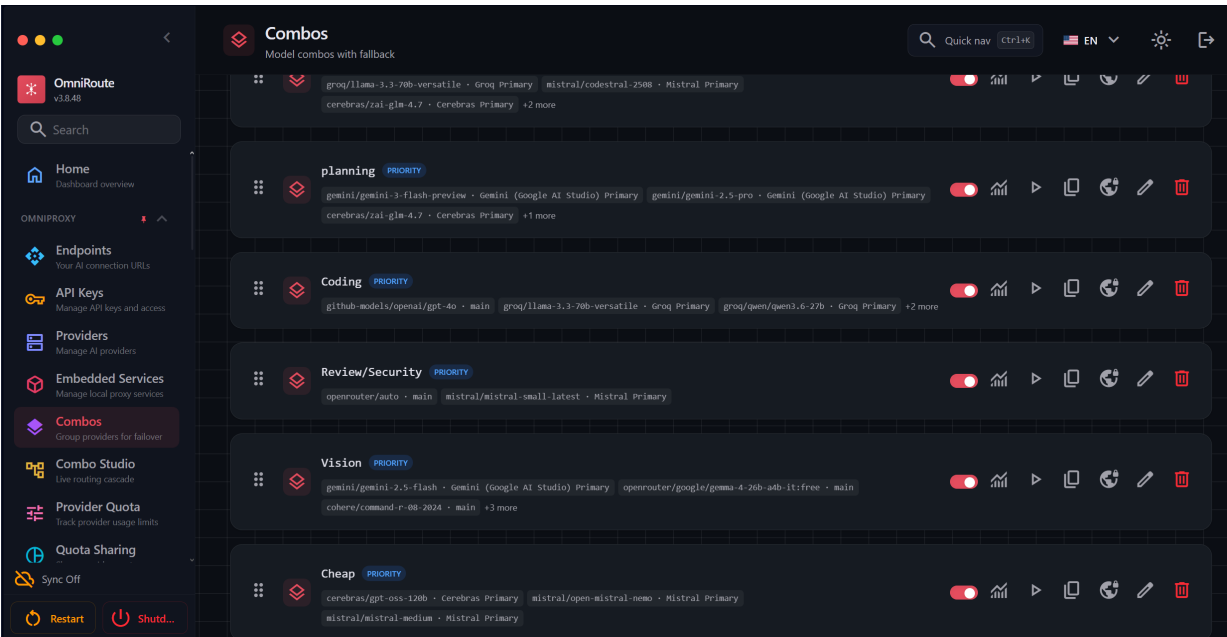
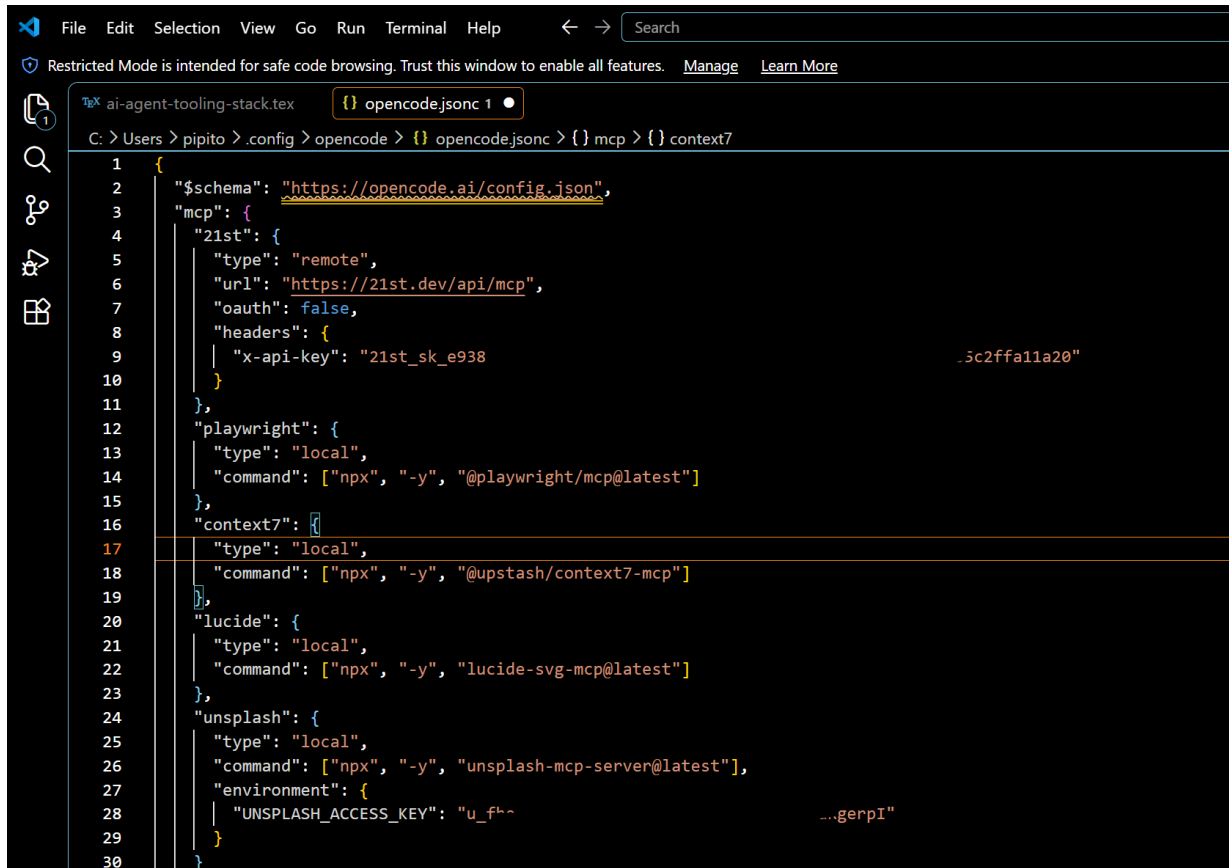


Figure 3: OmniRoute Combos establishing priority routing for different tasks.

The routing layer keeps callers decoupled from specific models: a pool can gain, lose or swap backends without touching client code — the same abstraction discipline used for services behind a load balancer in classical infrastructure.

4 Browser automation

Web-based workflows were evaluated and operationalized with browser-driving tools — **Playwright MCP** and **Browser Use** — giving agents reliable hands on real pages: navigation, extraction, form flows and end-to-end verification steps inside AI-assisted development loops.



```
1  {
2    "$schema": "https://opencode.ai/config.json",
3    "mcp": {
4      "21st": {
5        "type": "remote",
6        "url": "https://21st.dev/api/mcp",
7        "oauth": false,
8        "headers": {
9          | "x-api-key": "21st_sk_e938
10         }
11      },
12      "playwright": {
13        "type": "local",
14        "command": ["npx", "-y", "@playwright/mcp@latest"]
15      },
16      "context7": {
17        "type": "local",
18        "command": ["npx", "-y", "@upstash/context7-mcp"]
19      },
20      "lucide": {
21        "type": "local",
22        "command": ["npx", "-y", "lucide-svg-mcp@latest"]
23      },
24      "unsplash": {
25        "type": "local",
26        "command": ["npx", "-y", "unsplash-mcp-server@latest"],
27        "environment": {
28          | "UNSPLASH_ACCESS_KEY": "u_f~
29        }
30      }
31    }
32  }
```

Figure 4: MCP server configuration file integrating Playwright into the local environment.

5 CrewAI multi-agent framework

The orchestration layer is a hierarchical multi-agent system built on **CrewAI**: **ten specialist agents** under a **custom manager agent** with strict delegation-only behavior. The manager never does specialist work itself; it decomposes objectives, assigns them, and integrates results.



Figure 5: CrewAI multi-agent topology showing the Manager Agent delegating tasks to 10 specialists.

Model access flows through OmniRoute, so each role draws on the pool that fits its task — heavy reasoning for planning, cheap fast models for routine steps.

6 Why it matters

- Treats AI capability as capacity planning under hardware constraints, not magic.
- A routing layer with named pools is the difference between a pile of models and an operable platform.
- Delegation-only managers encode accountability into agent systems — clear ownership of every subtask.

7 Next steps

Documentation is intentionally thin today; planned follow-ups include publishing configuration notes for OmniRoute’s pool definitions, quantization benchmarks against the 8 GB VRAM budget, and a write-up of the CrewAI delegation patterns.